Modeling Stage Guideline — Ames Housing Price Prediction

Scope of this document: what variables to use, what to build, and what to deliver. This applies to **both** engineers working on this stage.

---

0. Which Data You Work On — Read This First

We have **two** preprocessed feature sets sitting at the end of the preprocessing notebook (`Ames_Housing_v7.ipynb`):

- ❌ **Reduced set (26 features → 60 columns) — NOT used in this stage.** Ignore `X_train_processed_reduced_final` / `X_val_processed_reduced_final` / `X_test_processed_reduced_final`. This was a side exploration, not part of the main deliverable.
- ✅ **Full pipeline (79 features → 167 columns) — this is what you both work on.**

The exact variables you will import and use

دي المتغيرات اللي هتدخل للموديل مباشرة

```
# Features (already imputed, encoded, scaled, and feature-selected)
X_train_selected_full   # shape (1875, 167)
X_val_selected_full     # shape (469, 167)
X_test_selected_full    # shape (585, 167)

# Target (already log-transformed — log1p(SalePrice))
y_train_log              # shape (1875,)
y_val_log                # shape (469,)
y_test_log                # shape (585,)
```

Do not re-run or modify the preprocessing pipeline. Import these variables as-is (either by re-running the preprocessing notebook up to the point they're created, or by loading them from a saved `.pkl`/`.csv` if we export them). No re-imputation, no re-encoding, no re-scaling, no re-splitting.

Important: the target is **log-transformed**. Whatever metric you report, always also report it back-transformed to real dollars (use `np.expm1()` on predictions before computing dollar-scale error) — a model's R^2 on `log(SalePrice)` is not intuitive to stakeholders; RMSE in dollars is.

---

1. Eng. Mohamed — SVR Track

1.1 What to build

A single **Support Vector Regression (SVR)** model trained on `X_train_selected_full` / `y_train_log`.

1.2 Required steps

1. **Baseline SVR** — fit a default `SVR()` first, get a baseline validation score before any tuning. This is your reference point to prove the tuning actually helped.
2. **Hyperparameter tuning — GridSearchCV**, over at minimum:

- `kernel`: `['linear', 'rbf', 'poly']`
- `C`: a log-scale range (e.g. `[0.1, 1, 10, 100]`)
- `epsilon`: e.g. `[0.01, 0.1, 0.5, 1]`
- `gamma`: e.g. `['scale', 'auto']` (relevant for `rbf`/`poly`)
- Use **`cv=5`** (or whatever the team's standard fold count is) and score on a metric appropriate for regression (`neg_root_mean_squared_error` or `r2`).

3. **Evaluate the tuned model** on `X_val_selected_full` (not the test set — test stays untouched until final comparison across all models).
4. Report metrics **on both the log scale and back-transformed dollar scale**:
   - RMSE, MAE, R^2 — log scale
   - RMSE, MAE — dollar scale (after `np.expm1()`)
5. **Feature importance / interpretation** (SVR-appropriate): if using a `linear` kernel, report coefficient magnitudes; for `rbf`/`poly`, note that direct feature importance isn't naturally available (mention this limitation rather than skipping it).
6. **Diagnostic plots**: predicted vs. actual scatter plot, residual plot — both on the dollar scale.

1.3 What Eng. Mohamed must deliver to me (for the Streamlit app)

- ☐ The **final tuned SVR model object**, saved (`joblib.dump(...)` or `pickle`) — this is the actual artifact the Streamlit app will load to make live predictions.
- ☐ The **best hyperparameters** found by GridSearchCV (as a dict/JSON).
- ☐ **Validation metrics table**: RMSE, MAE, R^2 (log scale) + RMSE, MAE (dollar scale).
- ☐ The **GridSearchCV results summary** (top few parameter combinations and their scores) — for the report, not necessarily for the app.
- ☐ The two diagnostic plots (predicted-vs-actual, residuals) as saved image files.
- ☐ A short written note (few lines) on what kernel/params won and why that makes sense given the data.

---

2. Eng. Habiba — Ensemble (XGBoost + Random Forest via Stacking) Track

2.1 What to build

Two base models trained on `X_train_selected_full` / `y_train_log`, then combined into a **Stacking Generalization** ensemble:

1. **Base model 1:** `RandomForestRegressor`
2. **Base model 2:** `XGBRegressor` (XGBoost)
3. **Meta-model (stacked on top):** a simple regressor trained on the *out-of-fold predictions* of the two base models (commonly `LinearRegression` or `Ridge` as the meta-learner — keep it simple, its job is just to learn how to weight/combine the two base models' outputs).

Recommended implementation: `sklearn.ensemble.StackingRegressor` (handles the out-of-fold prediction generation internally, avoiding leakage from the base models into the meta-model).

2.2 Required steps

1. **Tune each base model individually first** (before stacking):
   - `RandomForestRegressor`: tune `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features` via `GridSearchCV` or `RandomizedSearchCV` (Randomized is fine here — RF grids get large fast).
   - `XGBRegressor`: tune `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `reg_alpha`/`reg_lambda` via `GridSearchCV` or `RandomizedSearchCV`.
   - Use the same `cv` fold count as the SVR track for consistency.
2. **Build the `StackingRegressor`** using the two *tuned* base models, with `cv=5` (for generating clean out-of-fold predictions to train the meta-model on).
3. **Evaluate three things separately** on `X_val_selected_full`, so we can see the ensemble's actual value-add:
   - Tuned Random Forest alone

- Tuned XGBoost alone
- The full Stacking ensemble

4. Report metrics for **all three** — log scale and dollar scale (same as SVR track: RMSE, MAE, R^2).
5. **Feature importance**: report native feature importances from both RF and XGBoost (these are directly available — `.feature_importances_`). Show the top ~15–20 features for each, and note where they agree/disagree.
6. **Diagnostic plots**: predicted-vs-actual and residuals for the final stacked model (dollar scale).

2.3 What Eng. Habiba must deliver to me (for the Streamlit app)

- ☐ The **final `StackingRegressor` object** (already fit), saved via `joblib.dump(...)` — this is the artifact the Streamlit app will load.
- ☐ The two **individually tuned base models** (RF and XGBoost), saved separately too — in case the app wants to show/compare all three (RF alone / XGB alone / Stacked).
- ☐ **Best hyperparameters** for RF and for XGBoost (as dicts/JSON).
- ☐ **Validation metrics table** for all three variants (RF alone, XGB alone, Stacked) — RMSE, MAE, R^2 on both log and dollar scale.
- ☐ **Feature importance rankings** (top 15–20) for RF and XGBoost, as a table or saved chart.
- ☐ The diagnostic plots (predicted-vs-actual, residuals) for the final stacked model.
- ☐ A short written note on whether stacking actually beat the best individual base model, and by how much.

---

3. Common Rules for Both Tracks (Non-Negotiable)

1. **Do not touch `X_test_selected_full` / `y_test_log` during tuning.** Test set is reserved for the final, single, end-of-project comparison across every model (SVR, RF, XGB, Stacked). Tune and select only against the validation set.
2. **Do not re-run preprocessing.** Use `X_train_selected_full` / `X_val_selected_full` / `y_train_log` / `y_val_log` exactly as provided. If something looks wrong with them, flag it — don't silently re-derive your own version.
3. **Fix a random seed** (`random_state=42` or whatever we standardize on) everywhere it's accepted, so results are reproducible.
4. **Report metrics on both scales** (log and dollar) — dollar scale is what goes on the Streamlit dashboard, log scale is what's technically comparable across models trained on the log target.
5. **Save everything as files**, not just notebook cell outputs — I need actual `.pkl` files for the models, and a metrics summary (CSV/JSON/table) I can pull straight into the app. Notebook-only outputs that die when the kernel closes are not a deliverable.

---

4. Final Deliverables Checklist (What I Need for Streamlit)

Item	SVR (Mohamed)	Ensemble (Stacking)
Trained model file(s) (`.pkl`)	✅ 1 file (final SVR)	✅ 3 files (RF, XGB, Stacked)
Best hyperparameters (JSON/dict)	✅	✅ (RF + XGB separately)
Validation metrics (log + dollar scale)	✅	✅ (all 3 variants)
Predicted-vs-actual plot	✅	✅
Residual plot	✅	✅
Feature importance	✅ (if linear kernel) / noted limitation (if not)	✅ (RF + XGB)
Short written summary of results	✅	✅

Once both are in, the **test set** (`X_test_selected_full` / `y_test_log`) will be used once, across all models, for the final head-to-head comparison — that part is not each engineer's individual responsibility, it happens after both deliverables are in.